Numerical and simulation techniques in finance

Nicolas Langrené

Beijing Normal-Hong Kong Baptist University

2025 Fall Semester

1 Programming

A computer performs elementary operations

- addition `x + y`
- subtraction `x - y`
- assignment `x = 2`
- comparison `x > y`

A computer program describes a **sequence of operations**.
During code execution, the states of the variables **evolve dynamically**.

Originally, sequences of instructions were controlled by ***branching*** instructions, such as `GOTO` or `IF branching`

```
C     A weird program for calculating Pi written in Fortran.
C     From: Fink, D.G., Computers and the Human Mind, Anchor Books, 1966.

      PROGRAM PI
      DIMENSION TERM(100)
      N=1
   3  TERM(N)=((-1)**(N+1))*(4./(2.*N-1.))
      N=N+1
      IF (N-101) 3,6,6
   6  N=1
   7  SUM98 = SUM98+TERM(N)
      WRITE(*,28) N, TERM(N)
      N=N+1
      IF (N-99) 7, 11, 11
  11  SUM99=SUM98+TERM(N)
      SUM100=SUM99+TERM(N+1)
      IF (SUM98-3.141592) 14,23,23
  14  IF (SUM99-3.141592) 23,23,15
  15  IF (SUM100-3.141592) 16,23,23
  16  AV89=(SUM98+SUM99)/2.
      AV90=(SUM99+SUM100)/2.
      COMANS=(AV89+AV90)/2.
      IF (COMANS-3.1415920) 21,19,19
  19  IF (COMANS-3.1415930) 20,21,21
  20  WRITE(*,26)
      GO TO 22
  21  WRITE(*,27) COMANS
  22  STOP
  23  WRITE(*,25)
      GO TO 22
  25  FORMAT('ERROR IN MAGNITUDE OF SUM')
  26  FORMAT('PROBLEM SOLVED')
  27  FORMAT('PROBLEM UNSOLVED', F14.6)
```

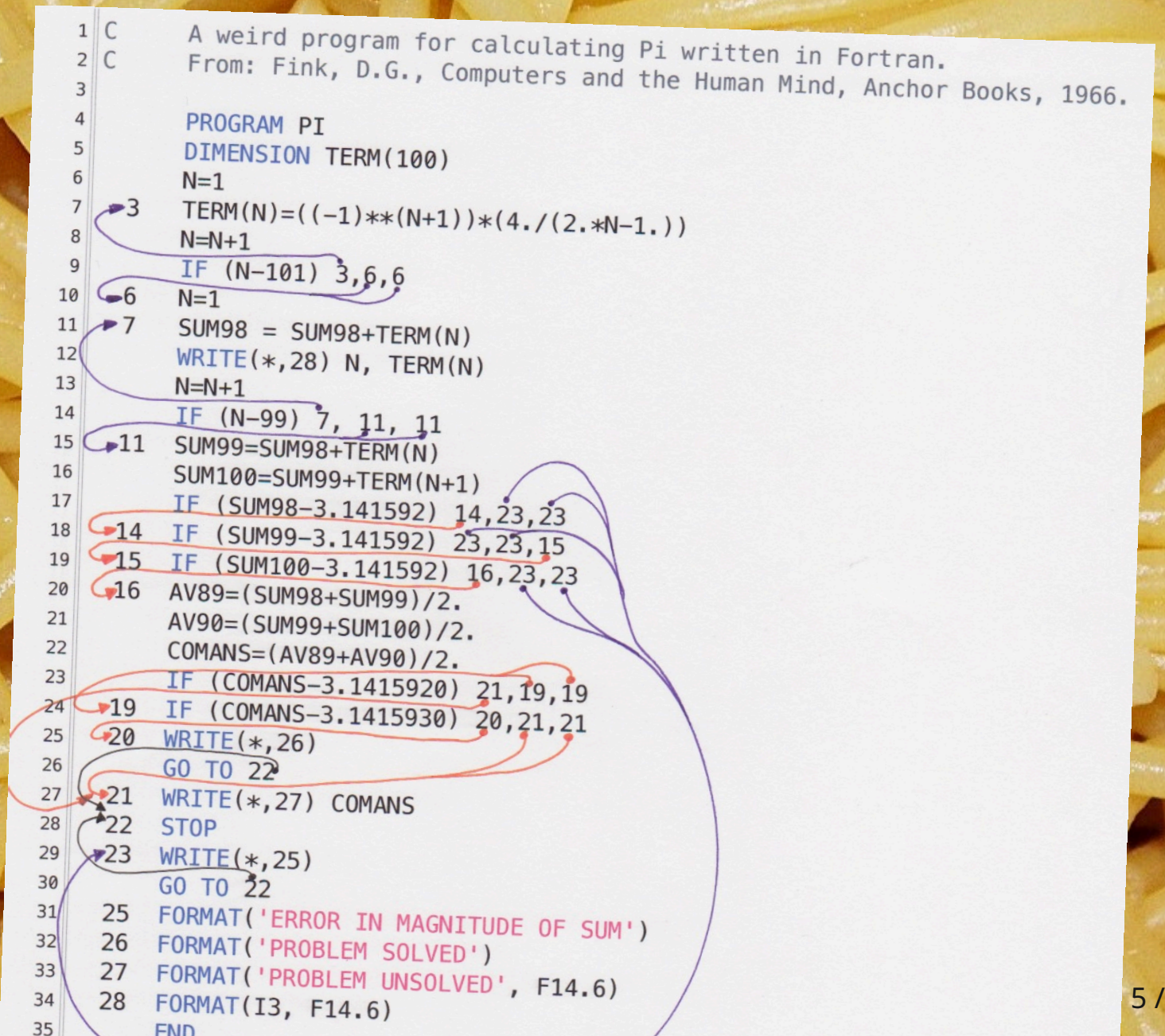
Originally, sequences of instructions were controlled by branching instructions, such as `GOTO` or `IF branching`

Spaghetti code

▷ Hard to understand how the variables evolve

▷ hard to maintain

▷ error-prone



```
C     A weird program for calculating Pi written in Fortran.
C     From: Fink, D.G., Computers and the Human Mind, Anchor Books, 1966.

      PROGRAM PI
      DIMENSION TERM(100)
      N=1
3     TERM(N)=((-1)**(N+1))*(4./(2.*N-1.))
      N=N+1
      IF (N-101) 3,6,6
6     N=1
7     SUM98 = SUM98+TERM(N)
      WRITE(*,28) N, TERM(N)
      N=N+1
      IF (N-99) 7, 11, 11
11    SUM99=SUM98+TERM(N)
      SUM100=SUM99+TERM(N+1)
      IF (SUM98-3.141592) 14,23,23
14    IF (SUM99-3.141592) 23,23,15
15    IF (SUM100-3.141592) 16,23,23
16    AV89=(SUM98+SUM99)/2.
      AV90=(SUM99+SUM100)/2.
      COMANS=(AV89+AV90)/2.
      IF (COMANS-3.1415920) 21,19,19
19    IF (COMANS-3.1415930) 20,21,21
20    WRITE(*,26)
      GO TO 22
21    WRITE(*,27) COMANS
22    STOP
23    WRITE(*,25)
      GO TO 22
25    FORMAT('ERROR IN MAGNITUDE OF SUM')
26    FORMAT('PROBLEM SOLVED')
27    FORMAT('PROBLEM UNSOLVED', F14.6)
28    FORMAT(I3, F14.6)
      END
```

Structured programming

After many catastrophic failures, a new trend of thought in favor of a new *programming style* eventually emerged

[1] E. Dijkstra. "Go To statement considered harmful". In: *Communications of the ACM* 11.3 (1968), pp. 147-148.

[2] M. Wilkes. "The outer and inner syntax of a programming language". In: *The Computer Journal* 11.3 (1968), pp. 260-263.

These two articles triggered a debate about the use of branching instructions in programming code.

Structured programming

The **structured programming** paradigm only allows these three structures:

- **sequence**
- **repetition**
- **choice**

Sequence

It is a simple, arbitrary long sequence of basic instructions such as assignments, mutations, reading, writing, etc.

```
read y
x = sqrt(y)
.
.
.
write z
```

Repetition (1/2)

A repetition of a block of instructions, also called **loop**, can take either of the following two forms:

```
for each i in the set I repeat
.
.   (instructions)
.
end
```

The number of repetitions is defined by the *number of elements* in the **set I**.

Repetition (2/2)

A repetition of a block of instructions, also called **loop**, can take either of the following two forms:

```
while condition is true repeat
.
.   (instructions)
.
end
```

The number of repetition is **infinite** *as long as the condition remains true*.

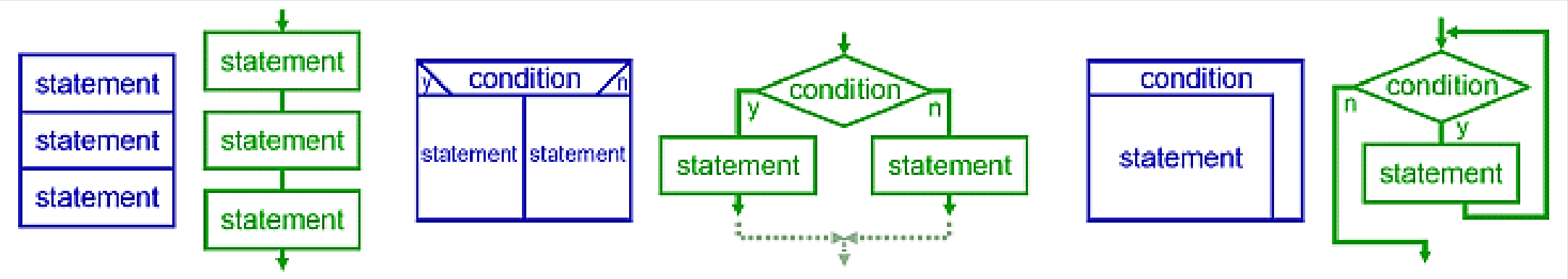
Choice

A choice between several possible blocks of instructions:

```
if condition 1 is true then
⋮   (instructions 1)
else if condition 2 is true then
⋮   (instructions 2)
⋮
else
⋮   (instructions)
end
```

The instructions following the first true condition are executed.

Hierarchical structure



▷ Any computer program is composed of a succession of these three structures, executed one after the other.

▷ Unlike programs which contain branching instructions, these structured programs can be read hierarchically from top to bottom, making them ***easier to understand*** and therefore ***less error prone***.

Programming paradigms

Following **structured programming**, several other programming paradigms have been proposed over the years. Two examples:

Object-oriented programming

Data and methods to manipulate them are kept as one unit called an object. Programs are designed around objects that interact with one another (C++, Java).

Functional programming

Programs are constructed by applying and composing functions. Functions can be passed as arguments and returned from other functions (R, JavaScript).

Most programming languages are **multi-paradigm**
(C++, Java, JavaScript, Python, R, ...)

1.1 Code examples

Python syntax for repetition and choice

```
# Definite number of repetitions
for i in set:
    instructions
```

```
# Indefinite number of repetitions
while expression:
    instructions
```

```
# Simple choice
if expression:
    instructions
```

```
# Multiple choice
if expression1:
    instructions1
elif expression2:
    instructions2
elif expression3:
    instructions3
else:
    instructions
```

Average of a vector

$$\bar{x} = \frac{1}{n}(x_1 + x_2 + \ldots + x_n) = \frac{1}{n}\sum_{i=1}^{n} x_i$$

```
n=5
x = np.random.randint(low=0, high=10, size=n)
print(x)
## [7 5 1 8 7]
```

```
s = 0
for i in np.arange(n):
    s = s + x[i]
sbar = s/n
print(sbar)
## 5.6
```

Machine precision

```
e = 1
while 1+e > 1:
    e = e/2
e = 2*e
print(e)
```

```
## 2.220446049250313e-16
```

Maximum of a vector

```
n = 5
x = np.random.randint(low=0, high=10, size=n)
print(x)
## [5 8 9 5 0]
```

```
m = 0
for i in np.arange(n):
    if x[i] > m:
        m = x[i]
print(m)
## 9
```

Bubble sort

```
print(x)
## [4 2 1 5 7 3]
swap = True
while swap:
    swap = False
    for i in np.arange(5):
        if x[i] > x[i+1]:
            swap = True
            temp = x[i+1]
            x[i+1] = x[i]
            x[i] = temp
print(x)
## [1 2 3 4 5 7]
```



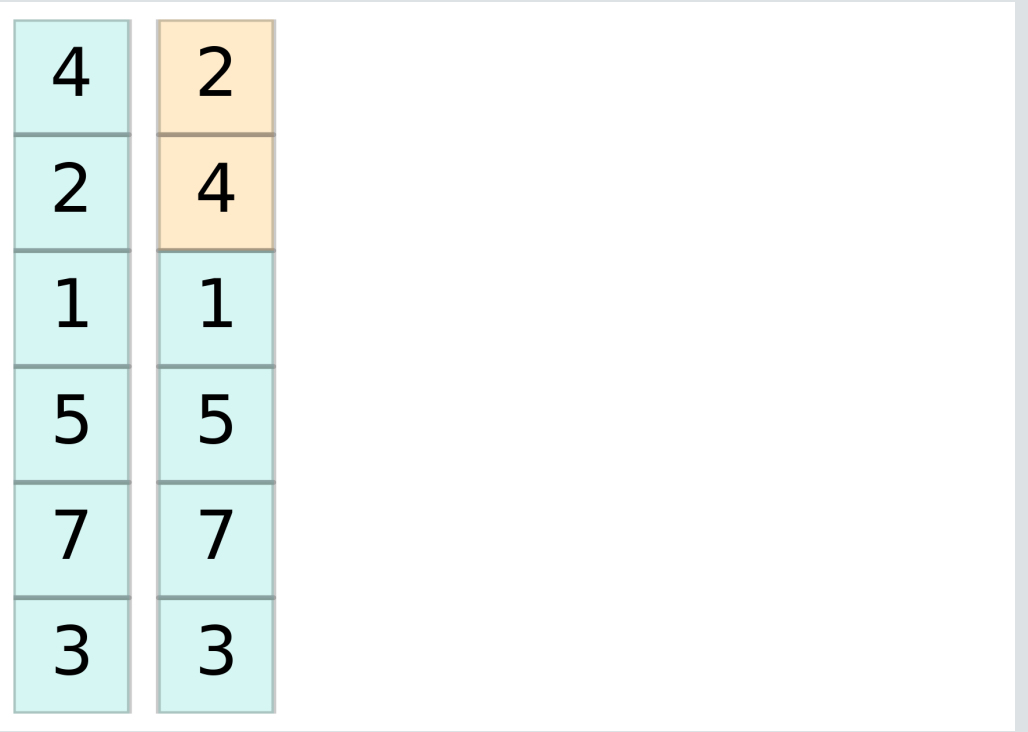
Bubble sort

```
print(x)
## [4 2 1 5 7 3]
swap = True
while swap:
    swap = False
    for i in np.arange(5):
        if x[i] > x[i+1]:
            swap = True
            temp = x[i+1]
            x[i+1] = x[i]
            x[i] = temp
print(x)
## [1 2 3 4 5 7]
```



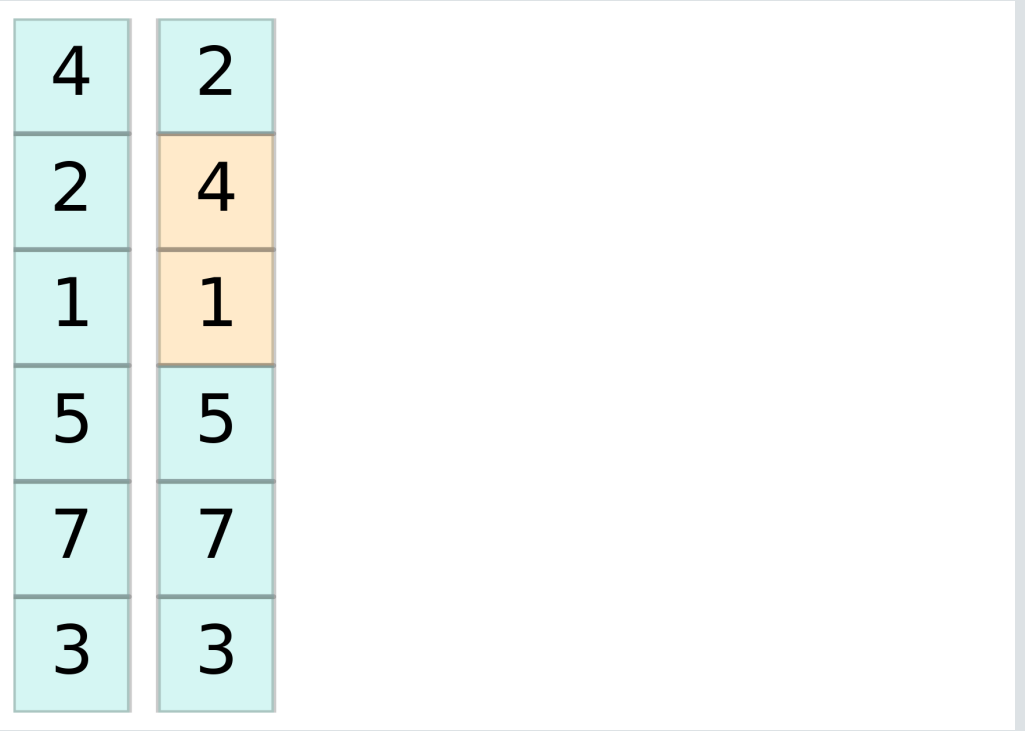
Bubble sort

```
print(x)
## [4 2 1 5 7 3]
swap = True
while swap:
    swap = False
    for i in np.arange(5):
        if x[i] > x[i+1]:
            swap = True
            temp = x[i+1]
            x[i+1] = x[i]
            x[i] = temp
print(x)
## [1 2 3 4 5 7]
```



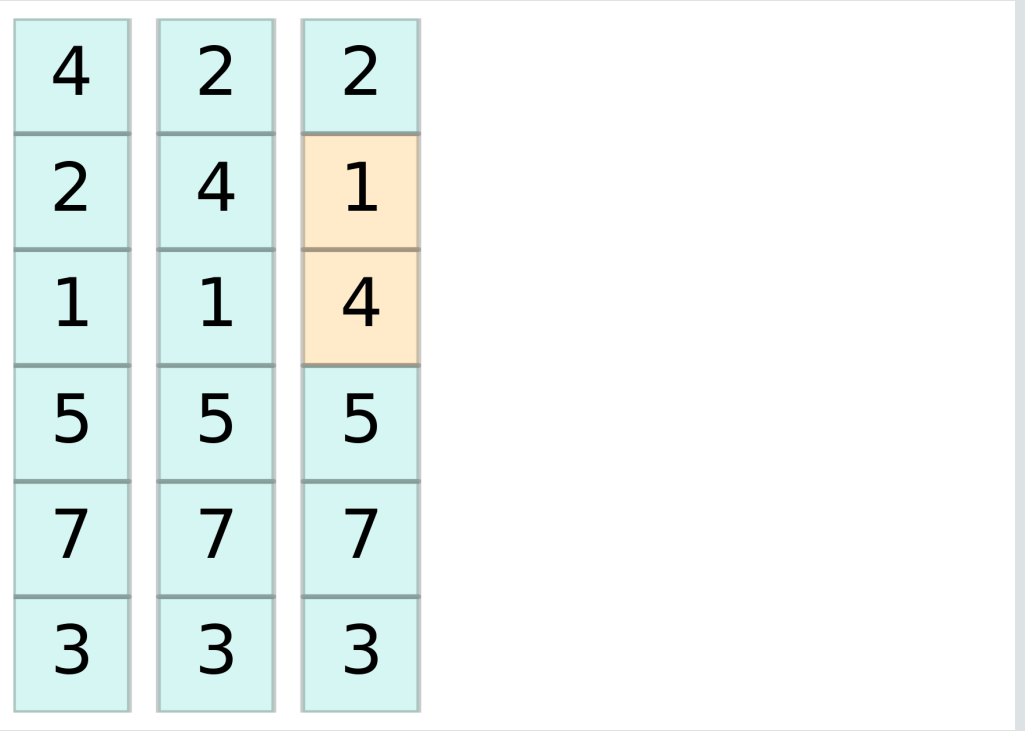
Bubble sort

```
print(x)
## [4 2 1 5 7 3]
swap = True
while swap:
    swap = False
    for i in np.arange(5):
        if x[i] > x[i+1]:
            swap = True
            temp = x[i+1]
            x[i+1] = x[i]
            x[i] = temp
print(x)
## [1 2 3 4 5 7]
```



Bubble sort

```
print(x)
## [4 2 1 5 7 3]
swap = True
while swap:
    swap = False
    for i in np.arange(5):
        if x[i] > x[i+1]:
            swap = True
            temp = x[i+1]
            x[i+1] = x[i]
            x[i] = temp
print(x)
## [1 2 3 4 5 7]
```



Bubble sort

```
print(x)
## [4 2 1 5 7 3]
swap = True
while swap:
    swap = False
    for i in np.arange(5):
        if x[i] > x[i+1]:
            swap = True
            temp = x[i+1]
            x[i+1] = x[i]
            x[i] = temp
print(x)
## [1 2 3 4 5 7]
```

4	2	2
2	4	1
1	1	4
5	5	5
7	7	7
3	3	3

Bubble sort

```
print(x)
## [4 2 1 5 7 3]
swap = True
while swap:
    swap = False
    for i in np.arange(5):
        if x[i] > x[i+1]:
            swap = True
            temp = x[i+1]
            x[i+1] = x[i]
            x[i] = temp
print(x)
## [1 2 3 4 5 7]
```

4	2	2	2
2	4	1	1
1	1	4	4
5	5	5	5
7	7	7	3
3	3	3	7

Bubble sort

```
print(x)
## [4 2 1 5 7 3]
swap = True
while swap:
    swap = False
    for i in np.arange(5):
        if x[i] > x[i+1]:
            swap = True
            temp = x[i+1]
            x[i+1] = x[i]
            x[i] = temp
print(x)
## [1 2 3 4 5 7]
```

4	2	2	2
2	4	1	1
1	1	4	4
5	5	5	5
7	7	7	3
3	3	3	7

Bubble sort

```
print(x)
## [4 2 1 5 7 3]
swap = True
while swap:
    swap = False
    for i in np.arange(5):
        if x[i] > x[i+1]:
            swap = True
            temp = x[i+1]
            x[i+1] = x[i]
            x[i] = temp
print(x)
## [1 2 3 4 5 7]
```

4	2	2	2	1
2	4	1	1	2
1	1	4	4	4
5	5	5	5	5
7	7	7	3	3
3	3	3	7	7

Bubble sort

```
print(x)
## [4 2 1 5 7 3]
swap = True
while swap:
    swap = False
    for i in np.arange(5):
        if x[i] > x[i+1]:
            swap = True
            temp = x[i+1]
            x[i+1] = x[i]
            x[i] = temp
print(x)
## [1 2 3 4 5 7]
```

4	2	2	2	1
2	4	1	1	2
1	1	4	4	4
5	5	5	5	5
7	7	7	3	3
3	3	3	7	7

Bubble sort

```
print(x)
## [4 2 1 5 7 3]
swap = True
while swap:
    swap = False
    for i in np.arange(5):
        if x[i] > x[i+1]:
            swap = True
            temp = x[i+1]
            x[i+1] = x[i]
            x[i] = temp
print(x)
## [1 2 3 4 5 7]
```



4	2	2	2	1	1
2	4	1	1	2	2
1	1	4	4	4	4
5	5	5	5	5	3
7	7	7	3	3	5
3	3	3	7	7	7

Bubble sort

```
print(x)
## [4 2 1 5 7 3]
swap = True
while swap:
    swap = False
    for i in np.arange(5):
        if x[i] > x[i+1]:
            swap = True
            temp = x[i+1]
            x[i+1] = x[i]
            x[i] = temp
print(x)
## [1 2 3 4 5 7]
```

4	2	2	2	1	1
2	4	1	1	2	2
1	1	4	4	4	4
5	5	5	5	5	3
7	7	7	3	3	5
3	3	3	7	7	7

Bubble sort

```
print(x)
## [4 2 1 5 7 3]
swap = True
while swap:
    swap = False
    for i in np.arange(5):
        if x[i] > x[i+1]:
            swap = True
            temp = x[i+1]
            x[i+1] = x[i]
            x[i] = temp
print(x)
## [1 2 3 4 5 7]
```

4	2	2	2	1	1	1
2	4	1	1	2	2	2
1	1	4	4	4	4	3
5	5	5	5	5	3	4
7	7	7	3	3	5	5
3	3	3	7	7	7	7

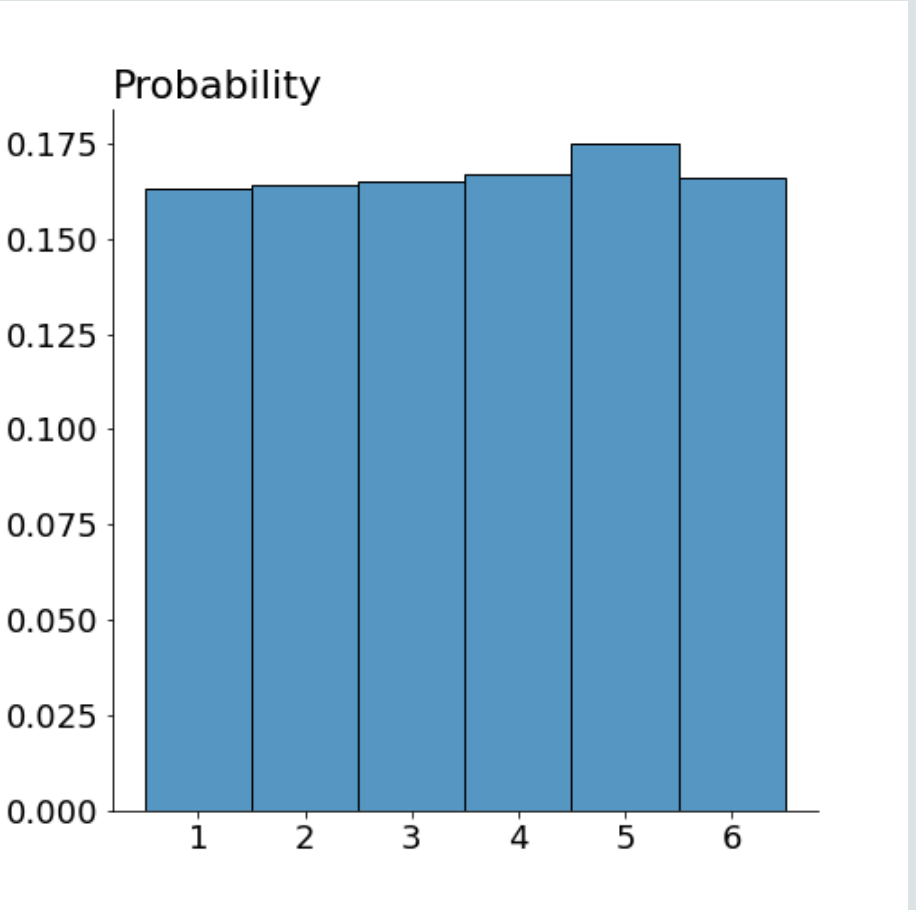
Bubble sort

```
print(x)
## [4 2 1 5 7 3]
swap = True
while swap:
    swap = False
    for i in np.arange(5):
        if x[i] > x[i+1]:
            swap = True
            temp = x[i+1]
            x[i+1] = x[i]
            x[i] = temp
print(x)
## [1 2 3 4 5 7]
```

4	2	2	2	1	1	1
2	4	1	1	2	2	2
1	1	4	4	4	4	3
5	5	5	5	5	3	4
7	7	7	3	3	5	5
3	3	3	7	7	7	7

Dice roll simulation

```
n = 1000
x = np.zeros(n)
for i in np.arange(n):
    u = np.random.rand() #random number in [0,1]
    if (u < 1/6):
        x[i] = 1
    elif (u < 2/6):
        x[i] = 2
    elif (u < 3/6):
        x[i] = 3
    elif (u < 4/6):
        x[i] = 4
    elif (u < 5/6):
        x[i] = 5
    else:
        x[i] = 6
sns.histplot(x,stat="probability",discrete=True)
```



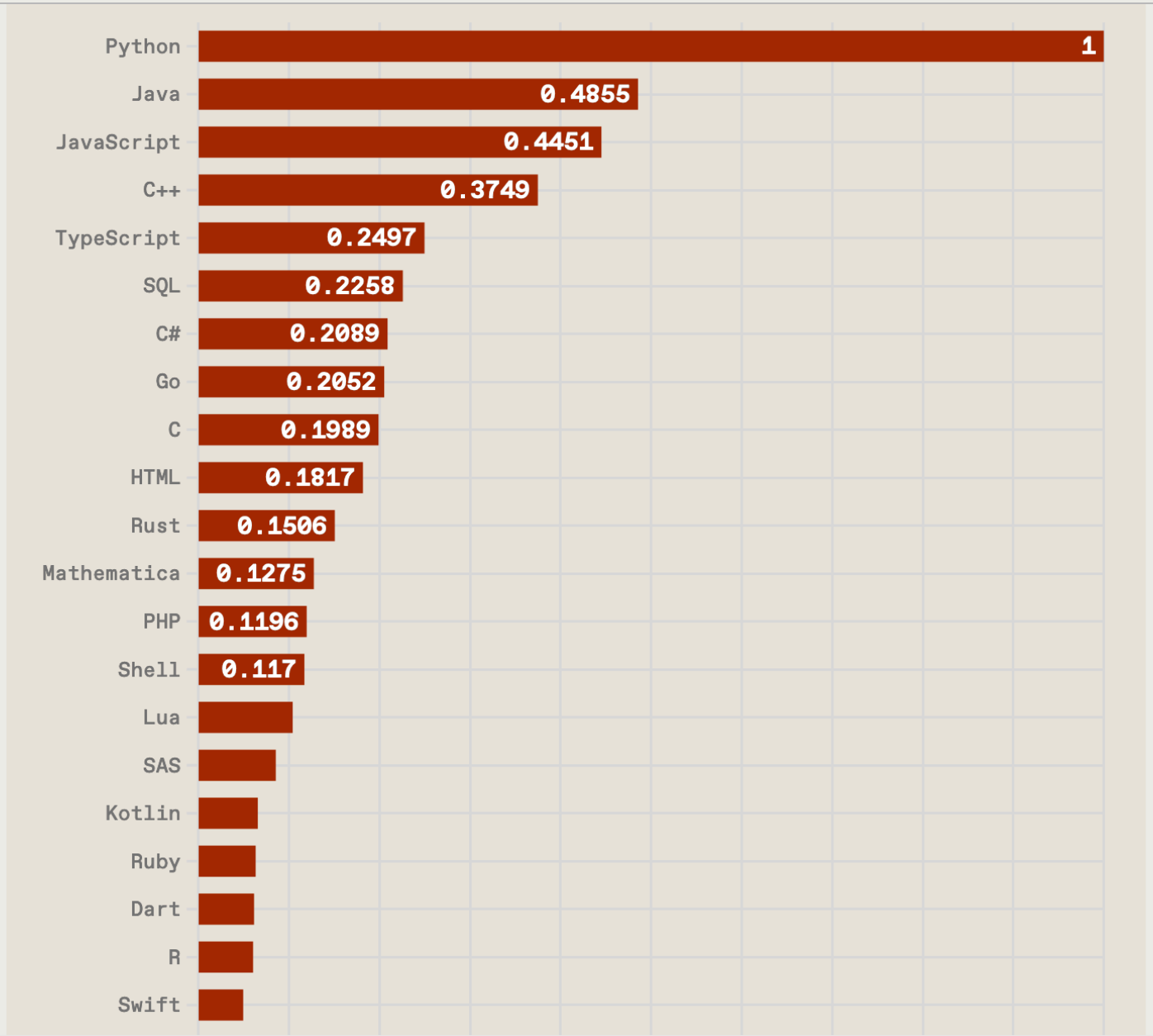
1.2 Introduction to Python

TAGS

TOP PROGRAMMING LA…

PYTHON | SQL | RUST

TYPESCRIPT



NumPy package

Python programming language https://docs.python.org/3/tutorial/
NumPy adds supports for vectors, matrices and multidimensional arrays
https://numpy.org/doc/stable/user/absolute_beginners.html
https://numpy.org/doc/stable/user/numpy-for-matlab-users.html

```
import numpy as np
x = np.array([8, 3, 7, 9])
x[0] = 12
x[3] = x[1] + x[2]
print(x)
## [12  3  7 10]
```

```
A = np.array([[4, 5], [7, 3]])
A[1,0] = A[0,0] - 2
print(A.T) # same as A.transpose()
## [[4 2]
##  [5 3]]
```

NumPy package

```
A = np.array([[4,5], [7,3]]) # A=[4, 5] , B=[2, 3]
B = np.array([[2,3], [4,1]]) #   [7, 3]     [4, 1]
```

```
# element-wise addition
A+B
# element-wise multiplication
A*B
# element-wise division
A/B
# element-wise power
A**2
# matrix multiplication
A @ B
# matrix power
np.linalg.matrix_power(A, 2)
```

```
## array([[ 6,  8],
##        [11,  4]])
## array([[ 8, 15],
##        [28,  3]])
## array([[2.        , 1.66666667],
##        [1.75      , 3.        ]])
## array([[16, 25],
##        [49,  9]])
## array([[28, 17],
##        [26, 24]])
## array([[51, 35],
##        [49, 44]])
```

Code examples revisited

```
x = np.array([8, 3, 7, 9])
np.mean(x) # vector average, same as np.average(x)
## 6.75
np.max(x) # maximum element of a vector
## 9
np.sort(x) # sorting a vector
## array([3, 7, 8, 9])
```

```
np.spacing(1) # machine precision, same as np.finfo(float).eps
## 2.220446049250313e-16
```

```
y = np.random.randint(1, 7, 1000) # dice throw simulation
```

NumPy array creation (1/2)

```
x0 = np.zeros(6) # array of size 6 filled with zeros
print(x0)
## [0. 0. 0. 0. 0. 0.]
y1 = np.ones(4) # array of size 4 filled with ones
print(y1)
## [1. 1. 1. 1.]
y0 = np.zeros_like(y1) # same shape as y1, filled with zeros
print(y0)
## [0. 0. 0. 0.]
x1 = np.ones_like(x0) # same shape as x0, filled with ones
print(x1)
## [1. 1. 1. 1. 1. 1.]
```

NumPy array creation (2/2)

```
x2 = np.repeat(2,7) # array of size 7 filled with twos
x3 = np.repeat(3,7) # array of size 7 filled with threes
print(x2)
## [2 2 2 2 2 2 2]
print(x3)
## [3 3 3 3 3 3 3]
x23 = np.vstack((x2,x3)) # vertical stack, size 2x7
print(x23)
## [[2 2 2 2 2 2 2]
##  [3 3 3 3 3 3 3]]
x323 = np.hstack((x3,x2,x3)) # horizontal stack, size 21
print(x323)
## [3 3 3 3 3 3 3 2 2 2 2 2 2 2 3 3 3 3 3 3 3]
```

NumPy array dimensions

```
x = np.arange(1,9,2) #grid between 1 and 9-1=8 with mesh size 2
print(x)
## [1 3 5 7]
y = np.linspace(0,8,9) # grid of 9 points from 0 to 8
print(y)
## [0. 1. 2. 3. 4. 5. 6. 7. 8.]
print(np.shape(x), ", ", np.size(x))
## (4,) ,  4
z = np.expand_dims(x,axis=1)
print(np.shape(z), ", ", np.size(z))
## (4, 1) ,  4
z = np.squeeze(z)
print(np.shape(z), ", ", np.size(z))
## (4,) ,  4
```

NumPy array repeat along axis

```
x = np.arange(4)
print( x )
## [0 1 2 3]
print( np.repeat(x,3) ) # repeating an array
## [0 0 0 1 1 1 2 2 2 3 3 3]
print( np.repeat( np.expand_dims(x,axis=0),3,axis=0) )
## [[0 1 2 3]
##  [0 1 2 3]
##  [0 1 2 3]]
print( np.repeat( np.expand_dims(x,axis=1),12,axis=1) )
## [[0 0 0 0 0 0 0 0 0 0 0 0]
##  [1 1 1 1 1 1 1 1 1 1 1 1]
##  [2 2 2 2 2 2 2 2 2 2 2 2]
##  [3 3 3 3 3 3 3 3 3 3 3 3]]
```

NumPy array reshape

```
x = np.array( [[2, 3, 4],
               [1, 5, 2]] )
print( np.reshape(x, (1,6)) )
## [[2 3 4 1 5 2]]
print( np.reshape(x, (3,2)) )
## [[2 3]
##  [4 1]
##  [5 2]]
print( np.reshape(x, (-1,1)) ) # same as (6,1)
## [[2]
##  [3]
##  [4]
##  [1]
##  [5]
##  [2]]
```

NumPy array maximum along axis

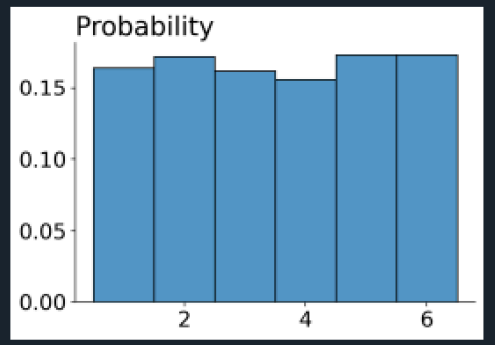
```
# element-wise maximum between two arrays
np.maximum([2, 3, 4], [1, 5, 2])
## array([2, 5, 4])
# same using max of matrix along rows
np.max([[2, 3, 4], [1, 5, 2]], axis=0)
## array([2, 5, 4])
np.max([[2, 3, 4], [1, 5, 2]], axis=1) #max along columns
## array([4, 5])
np.max([[2, 3, 4], [1, 5, 2]]) #max value of array
## 5
np.max( [[2, 3, 4],
         [1, 5, 2],
         [3, 4, 1]], axis=0)
## array([3, 5, 4])
```

Spyder (Python 3.9)

...Users/nlangrene/MEGA/WORK/TEACHING/2022_numerical_and_simulation_methods_in_finance/python/syntax.py

syntax.py | dice.py

```
11  x[0] = 12
12  x[3] = x[1] + x[2]
13
14  A = np.array([[4, 5], [7, 3]])
15  A[1,0] = A[0,0] - 2
16  print(A.T) # same as A.transpose()
17
18
19  print(A)
20  B = np.array([[2,3], [4,1]])
21  print((A,B))
22
23  A+B # element-wise addition
24  A*B # element-wise multiplication
25  A/B # element-wise division
26  A**2 # element-wise power
27
28  A @ B # matrix multiplication
29  np.linalg.matrix_power(A, 2) # matrix power
30
31
32  # Code examples revisited
33  # average of a vector
34  x = np.array([8, 3, 7, 9])
35  np.mean(x) # same as np.average(x)
36  # machine precision
37  np.spacing(1) # same as np.finfo(float).eps
38  np.max(x) # maximum element of a vector
39  np.sort(x) # sorting
40
41  # dice throw simulation
42  x = np.random.randint(1, 7, 1000)
43  import seaborn as sns
44  import matplotlib as mpl
45  import matplotlib.pyplot as plt
46  mpl.rcParams.update({'font.size': 20, 'figure.dpi': 200})
47  rc={'font.size': 22, 'axes.labelsize': 22, 'legend.fontsize': 22,
48      'axes.titlesize': 22, 'xtick.labelsize': 18, 'ytick.labelsize': 18}
49  plt.rcParams.update(**rc)
50
51  sns.histplot(x,stat="probability",discrete=True)
52  sns.despine()
53  plt.ylabel("")
54  plt.title("Probability", loc='left')
55  plt.show()
```



Help | Variable Explorer | Plots | Files

Console 1/A

```
 [2 3]]
(array([[4, 5],
        [2, 3]]), array([[2, 3],
        [4, 1]]))

In [141]: runfile('/Users/nlangrene/MEGA/WORK/TEACHING/
2022_numerical_and_simulation_methods_in_finance/python/
syntax.py', wdir='/Users/nlangrene/MEGA/WORK/TEACHING/
2022_numerical_and_simulation_methods_in_finance/
python')
[[4 2]
 [5 3]]
[[4 5]
 [2 3]]
(array([[4, 5],
        [2, 3]]), array([[2, 3],
        [4, 1]]))

In [142]:
```

IPython console | History

LSP Python: ready | conda: base (Python 3.9.12) | Line 50, Col 1 | UTF-8 | LF | RW | Mem 60%

Simple spectral analysis

An illustration of the Discrete Fourier Transform using windowing, to reveal the frequency content of a sound signal.

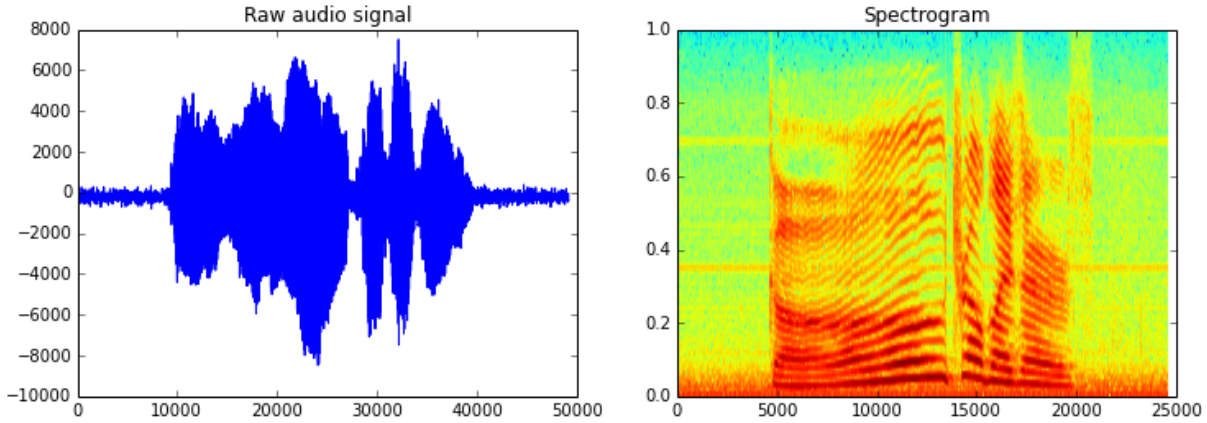
$$X_k = \sum_{n=0}^{N-1} x_n e^{-\frac{2\pi i}{N} kn} \qquad k = 0, \ldots, N-1$$

We begin by loading a datafile using SciPy's audio file support:

In [1]:
```
from scipy.io import wavfile
rate, x = wavfile.read('test_mono.wav')
```

And we can easily view its spectral structure using matplotlib's builtin `specgram` routine:

In [2]:
```
%matplotlib inline
from matplotlib import pyplot as plt
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4))
ax1.plot(x); ax1.set_title('Raw audio signal')
ax2.specgram(x); ax2.set_title('Spectrogram');
```

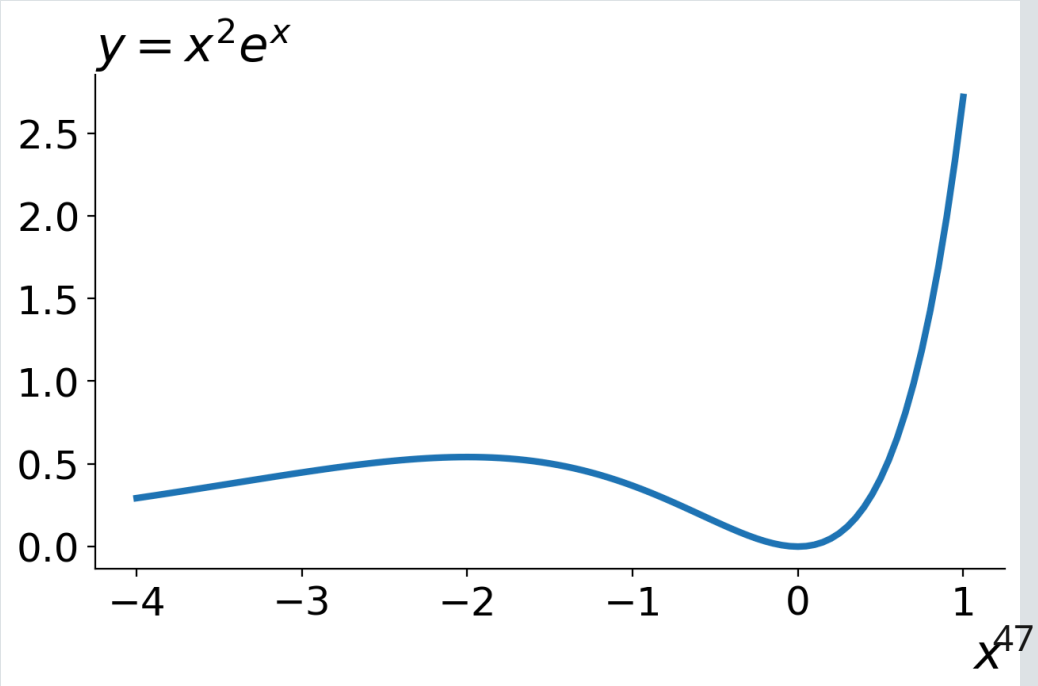


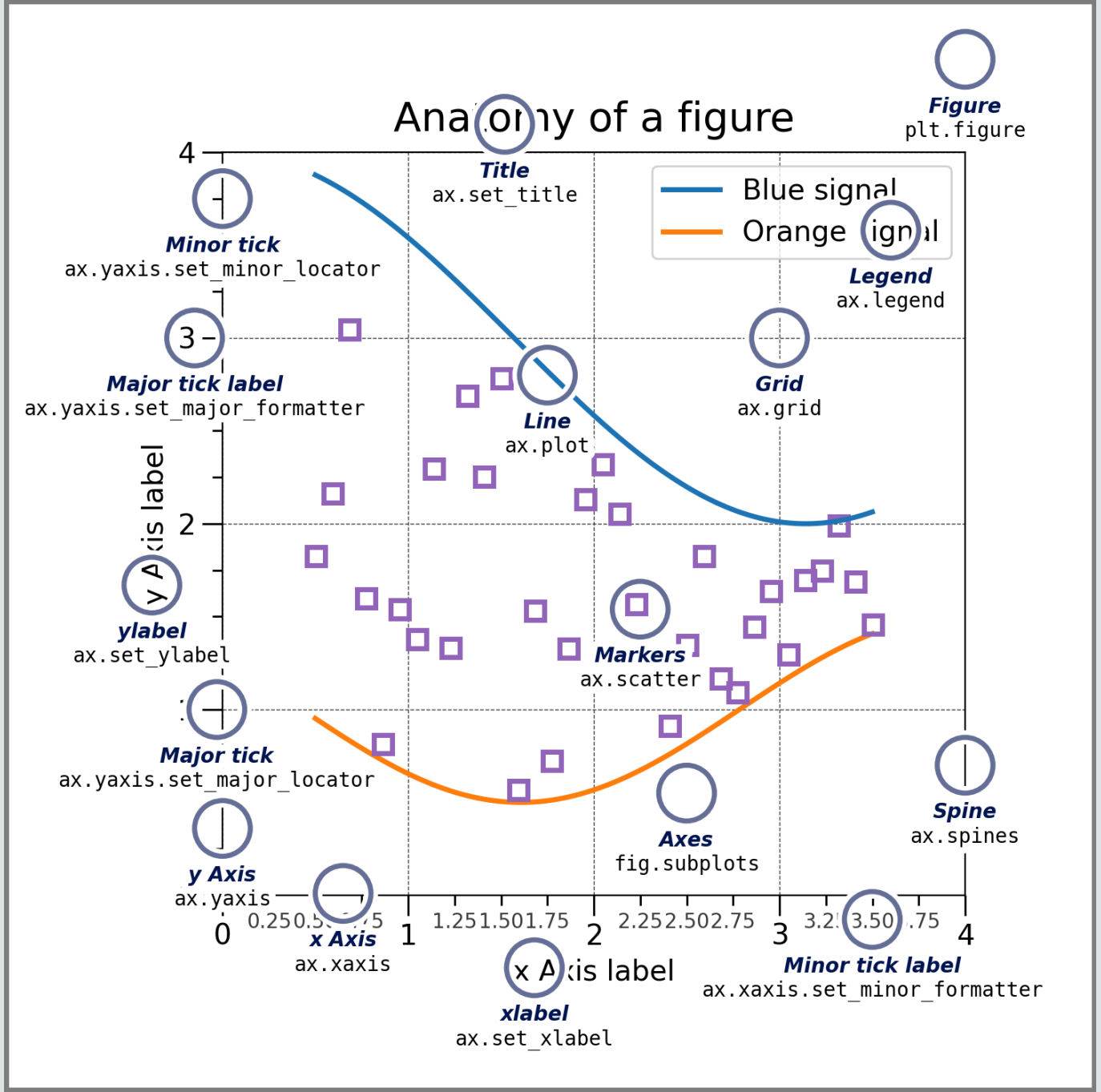
2D plots: Matplotlib

https://matplotlib.org/stable/tutorials/index

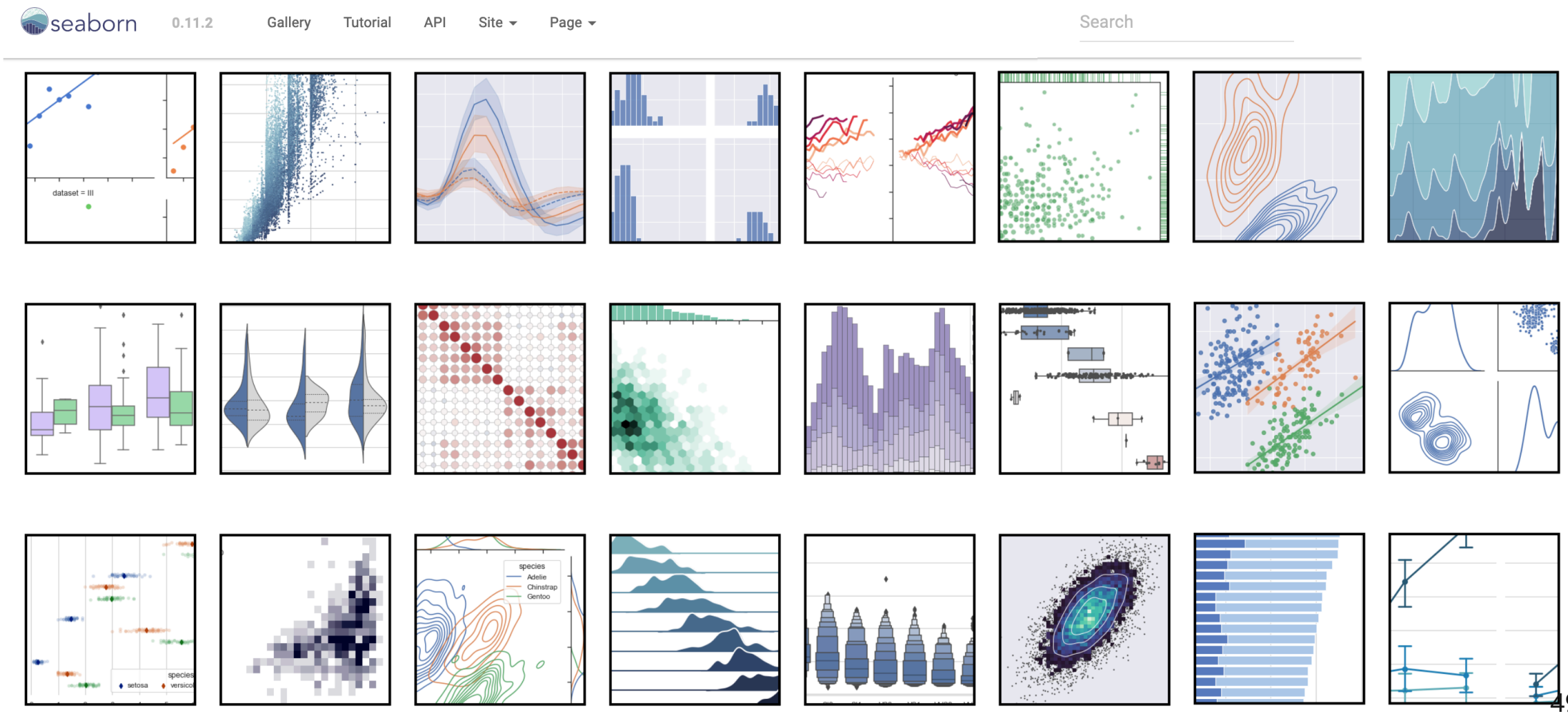
$$y = x^2 e^x$$

```
import matplotlib.pyplot as plt
x = np.linspace(-4,1,101)
y = x * x * np.exp(x)
fig, ax = plt.subplots()
ax.plot(x,y,linewidth=3)
ax.set_title('$y=x^2e^x$', loc=
ax.set_xlabel('$x$',loc='right')
plt.show()
```




Anatomy of a figure
Figure
plt.figure
Title
ax.set_title
Minor tick
ax.yaxis.set_minor_locator
Blue signal
Orange signal
Legend
ax.legend
Major tick label
ax.yaxis.set_major_formatter
Grid
ax.grid
Line
ax.plot
y Axis label
ylabel
ax.set_ylabel
Markers
ax.scatter
Major tick
ax.yaxis.set_major_locator
Axes
fig.subplots
Spine
ax.spines
y Axis
ax.yaxis
x Axis
ax.xaxis
x Axis label
xlabel
ax.set_xlabel
Minor tick label
ax.xaxis.set_minor_formatter

2D plots: Seaborn



3D plots

$$z = x^3 - 3xy^2$$

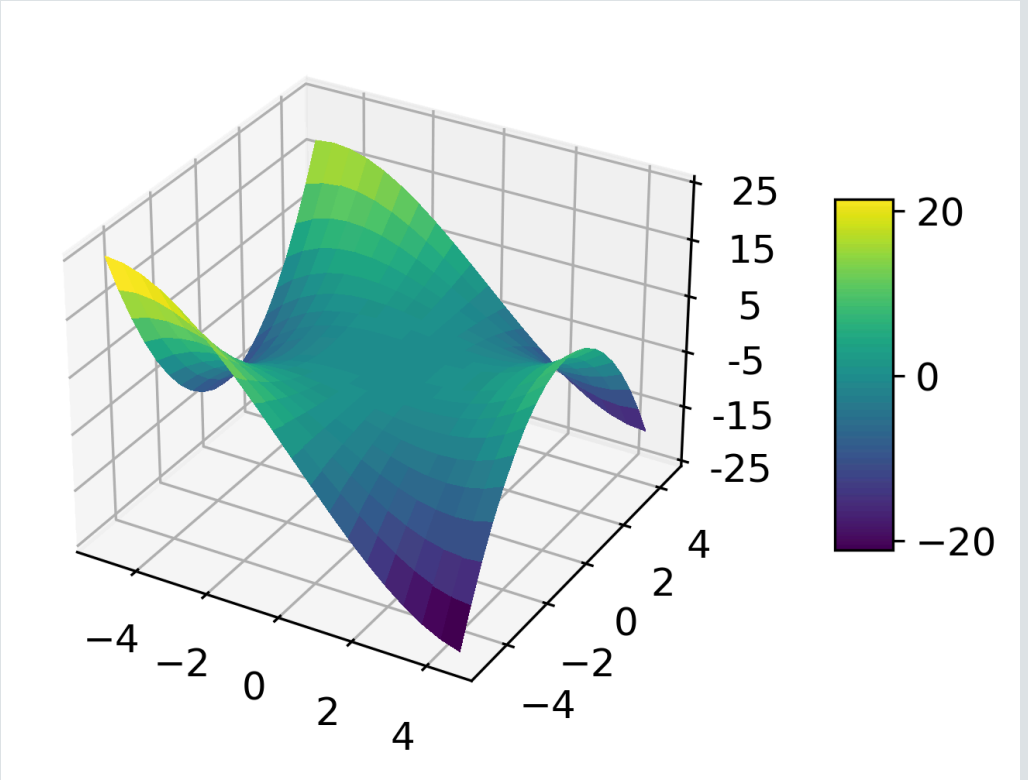
```
import numpy as np
x = np.arange(0, 5)
y = np.arange(0, 5)
X, Y = np.meshgrid(x, y)
Z = X*(X*X - 3*Y*Y)

print(Z)
## [[   0    1    8   27   64]
##  [   0   -2    2   18   52]
##  [   0  -11  -16   -9   16]
##  [   0  -26  -46  -54  -44]
##  [   0  -47  -88 -117 -128]]
```

```
print(X)
## [[0 1 2 3 4]
##  [0 1 2 3 4]
##  [0 1 2 3 4]
##  [0 1 2 3 4]
##  [0 1 2 3 4]]
print(Y)
## [[0 0 0 0 0]
##  [1 1 1 1 1]
##  [2 2 2 2 2]
##  [3 3 3 3 3]
##  [4 4 4 4 4]]
```

3D plots: Matplotlib

```
import numpy as np
fig,ax=plt.subplots(subplot_kw={"projectio
# Make data
X = np.arange(-5, 5, 0.5)
Y = np.arange(-5, 5, 0.5)
X, Y = np.meshgrid(X, Y)
Z = X*(X*X - 3*Y*Y) / 10
# Plot the surface
from matplotlib import cm # colormap
surf = ax.plot_surface(X, Y, Z,
       cmap=cm.viridis,linewidth=0,antiali
fig.colorbar(surf, shrink=0.5, aspect=6, p
plt.show()
```



Functions

```
def add(x,a=1):
    x = x+a
    return x

x = 1
y = add(x)
print((x,y))
## (1, 2)
y = add(x,2)
print((x,y))
## (1, 3)
```

https://docs.python.org/3/tutorial/controlflow.html#defining-functions

```
# Fibonacci series up to n
def fib(n):
    a, b = 0, 1
    while a < n:
        print(a, end=' ')
        a, b = b, a+b
    print()
fib(500)
## 0 1 1 2 3 5 8 13 21 34 55 89 144 233 377
```

Passing by assignment (1/2)

In Python, parameters are always **passed by assignment** to functions. These two programs are identical:

```
x = 25 # immutable object
def addten(v):
# v is assigned
# a new object: 35
    v += 10
    return v
print(addten(x))
## 35
print(x) # no side-effect
## 25
```

```
x = 25 # immutable object
v = x
# v is assigned
# a new object: 35
v += 10
print(v)
## 35
print(x) # no side-effect
## 25
```

Passing by assignment (2/2)

This means that understanding whether a function has side-effects boils down to understanding ***assignments***. These two programs are identical:

```
import numpy as np
# mutable object
x = np.array([12, 34, 55])
def addten(v,i):
    v[i] += 10 # v is modified
    return v
print(addten(x,-1))
## [12 34 65]
print(x) # side-effect
## [12 34 65]
```

```
import numpy as np
# mutable object
x = np.array([12, 34, 55])
v = x
v[-1] = 65 # v is modified
print(v)
## [12 34 65]
print(x) # side-effect
## [12 34 65]
```

Copy of an array

Assigning a mutable object such as an array to another one does not make a copy (the same object is shared). To force a copy, use **np.copy**

```
import numpy as np
x = np.array([11])
y = x
y[0] += 10
print( (x[0], y[0]) ) # x was modified
## (21, 21)

x = np.array([11])
y = np.copy(x) # USE COPY EXPLICITLY
y[0] += 10
print( (x[0], y[0]) ) # x was not modified
## (11, 21)
```

Deep copy of an array

If an array contains mutable objects, **np.copy** does not copy them (shallow copy). To force a copy of inner objects, use **cp.deepcopy**

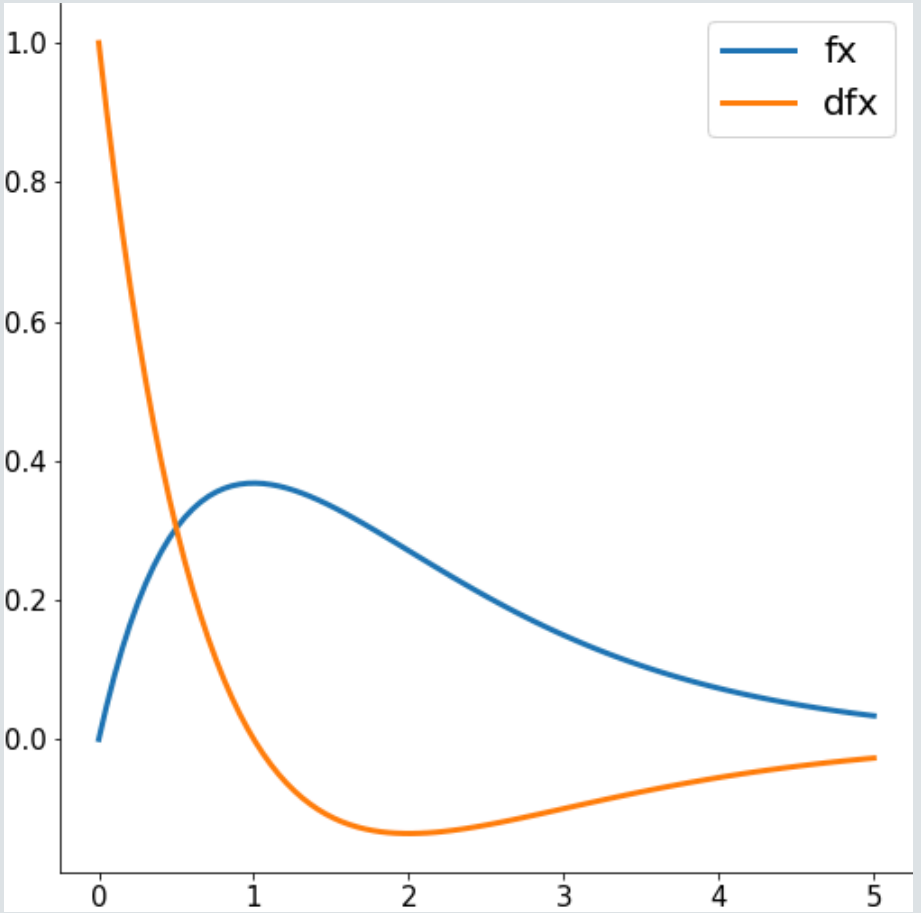
```
import numpy as np
x = np.array([1, 'm', [2,3,4]], dtype = object)
y = np.copy(x) # SHALLOW COPY
y[2][0] = 10
print(x)
## [1 'm' list([10, 3, 4])]
print(y)
## [1 'm' list([10, 3, 4])]

import copy as cp
x = np.array([1, 'm', [2,3,4]], dtype = object)
y = cp.deepcopy(x) # DEEP COPY
y[2][0] = 10
print(x)
## [1 'm' list([2, 3, 4])]
print(y)
## [1 'm' list([10, 3, 4])]
```

Lambda functions

```
square = lambda x: x*x
print(square(5))
## 25
(lambda x,y,z : x*y*z)(3,5,7)
## 105
def dnum(f, X):
    h = 0.001
    return (X, f(X),
    (f(X+h)-f(X-h))/(2*h) )

(x, fx, dfx) = \
dnum(lambda x: x/np.exp(x),
     np.linspace(0,5,100))
```



Black-Scholes formula

$$\text{Call} = S_0\mathcal{N}(d_1) - Ke^{-rT}\mathcal{N}(d_2)$$
$$\text{Put} = Ke^{-rT}\mathcal{N}(-d_2) - S_0\mathcal{N}(-d_1)$$

$$d_1 = \frac{\log(S_0/K)+(r+\sigma^2/2)T}{\sigma\sqrt{T}}$$
$$d_2 = d_1 - \sigma\sqrt{T}$$

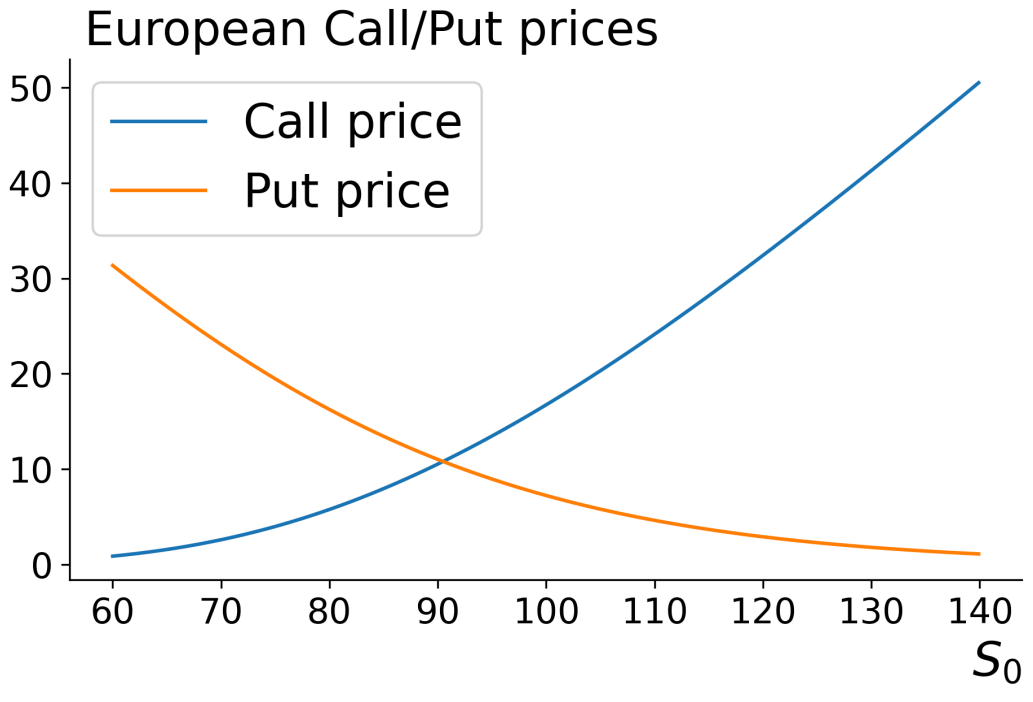
```
import numpy as np
from scipy.stats import norm
N = norm.cdf

def BS_CALL(S0, K, T, r, sigma):
    d1 = (np.log(S0/K) + (r + sigma**2/2)*T) / (sigma*np.sqrt(T))
    d2 = d1 - sigma * np.sqrt(T)
    return S0 * N(d1) - K * np.exp(-r*T)* N(d2)

def BS_PUT(S0, K, T, r, sigma):
    d1 = (np.log(S0/K) + (r + sigma**2/2)*T) / (sigma*np.sqrt(T))
    d2 = d1 - sigma* np.sqrt(T)
    return K*np.exp(-r*T)*N(-d2) - S0*N(-d1)
```

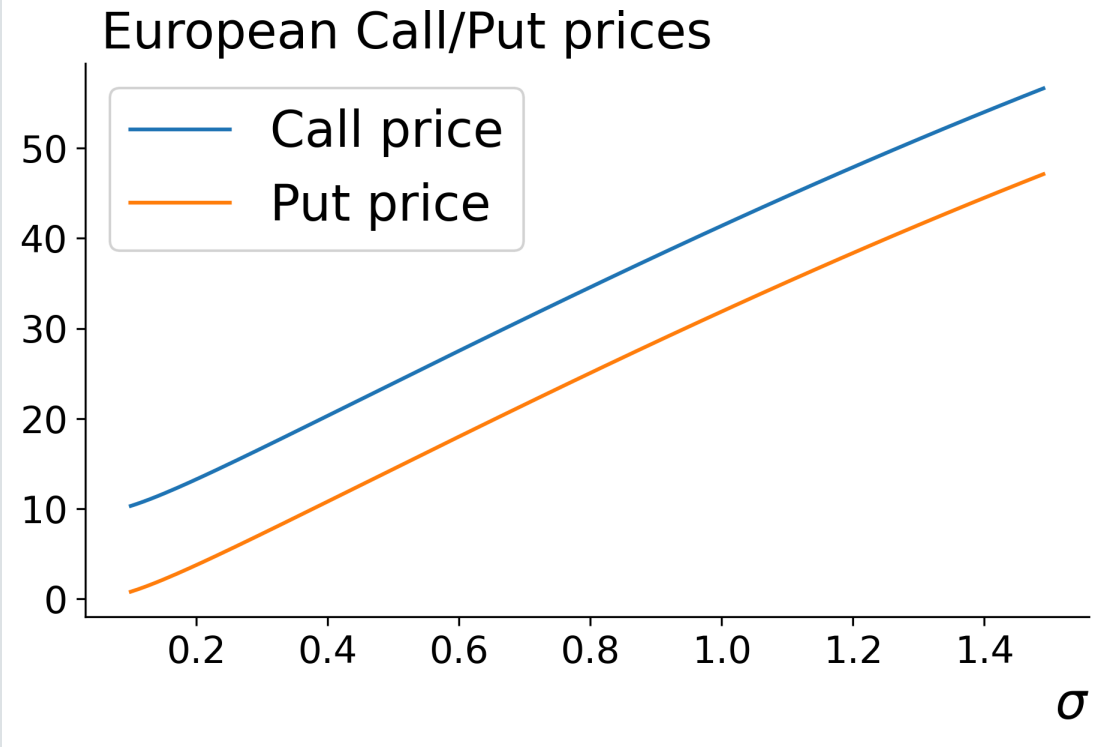
```
K, r, T, sigma = 100, 0.1, 1, 0.3
S0 = np.arange(60,140,0.1)
calls=[BS_CALL(s, K, T, r, sigma) for s in S0]
puts = [BS_PUT(s, K, T, r, sigma) for s in S0]

fig, ax = plt.subplots(1)
ax.plot(S0, calls, label='Call price')
ax.plot(S0, puts, label='Put price')
```




```
S0, K, r, T, = 100, 100, 0.1, 1
vols = np.arange(0.1, 1.5, 0.01)
calls=[BS_CALL(S0, K, T, r, sig) for sig in vols]
puts = [BS_PUT(S0, K, T, r, sig) for sig in vols]

fig, ax = plt.subplots(1)
ax.plot(vols, calls, label='Call price')
ax.plot(vols, puts, label='Put price')
```



Speeding-up computations: Vectorization

https://numpy.org/doc/stable/user/whatisnumpy.html#why-is-numpy-fast

```
from time import time
S0 = np.linspace(start=20,stop=180,num=1000000) # large array
```

```
# for loop
startTime = time()
calls1 = [BS_CALL(s, K, T, r, sigma) for s in S0] # DON'T USE LISTS AND FOR LOOPS
print('Execution time in seconds: ' + str(time() - startTime))
## Execution time in seconds: 315.156503
```

```
# vectorized code
startTime = time()
calls2 = BS_CALL(S0, K, T, r, sigma) # BETTER TO USE ARRAYS
print('Execution time in seconds: ' + str(time() - startTime))
## Execution time in seconds: 0.147062    # 2000 times faster
```

```
print(np.array_equal(calls1,calls2))
## True
```